1/9/26

Machine Learning

P. Sree Vaishnavi
192412219
IT406G4

Concept Learning + Candidate Elimination + Decision Tree

Safe Drinking water Quality Screening using Concept Learning and Decision Trees.

1. Problem statement:

A local community water board needs a transparent Machine learning screening tool to classify a water Sample as safe for use or Needs Treatment before detailed laboratory confirmation.

Instances: individual water Samples.

Target concept: SAFE class Labels: safe / Needs Treatment.

* Hypothesis representation for Candidate Elimination:

<PH, Turbidity, chlorine Microbial > "?" means any value and ∅ means no value yet. The final python model uses all six features.

Attribute	Possible Values	Purpose
PH band	Acidic / Neutral / Alkaline	coarse PH category
Turbidity	Low / high	cloudiness band
TDS band	Low / Medium / high	Dissolved-solids band
Residual chlorine	Adequate / Low	chlorine band
Microbial indicator	Absent / present	Synthetic indicator
Source Type	protected / community	Source category

2. Candidate Elimination / Version space

Initial boundaries : $S_0 = \langle \phi, \phi, \phi \rangle$ & $G_0 = \langle ?, ?, ?, ? \rangle$ ★

The following ten manually selected examples show how the boundaries become more specific as positive & negative evidence arrives.

Ex.	PH	Turb.	chlor.	Microbe	class
1	Neutral	Low	Adequate	Absent	safe
2	Neutral	high	Adequate	Absent	Needs Treatement
3	Acidic	Low	Adequate	Absent	Needs Treatment
4	Neutral	Low	Low	Absent	Needs Treatment
5	Neutral	Low	Adequate	present	Needs Treatment
6	Neutral	Low	Adequate	Absent	safe
7	Neutral	Low	Adequate	present	Needs Treatment
8	Neutral	High	Low	Absent	Needs Treatment
9	Alkaline	Low	Adequate	Absent	Needs Treatment
10	Neutral	Low	Adequate	Absent	Safe

S boundary

<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>

G boundary

<?, ?, ?, ?>
<?, low, ?, ?>
<Neutral, low, ?, ?>
<Neutral, low, Adequate, ?>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>
<Neutral, low, Adequate, Absent>

final boundary: $S = G_1 = \langle$ Neutral, Low, Adequate, Absent $\rangle$

★ Hence the simplified SAFE concept requires all four conditions.

3. Inductive Bias & Possible Error

* The inductive bias assumes that a useful water-safety concept can be represented as a conjunction of Categorical Attribute tests. This makes the model transparent.
* Example of bias causing an error: a sample can satisfy the 4 candidate-Elimination conditions while having high TDS or a surface source.

Lesson:

Consistency with a small training set does not prove real-world safety: The hypothesis language & training examples control the learned conclusion.

4. Decision Tree Construction

Entropy: $H(E) = -\sum p_i \log_2(P_i)$

Information Gain: $IG(E,A) = H(E) = \sum_v (|E_v|/|E|) H(E_v)$

The training set has 198 records: 34 safe & 158 Needs Treatment.

$\therefore H(S) = 0.6737$ bits

Candidate root	Information Gain
PH_Band	0.1038
Turbidity	0.0737
TDS_Band	0.0296
Residual_chlorine	0.0743
Microbial_indicator	0.0763
Source_Type	0.0482

Root Attribute: PH-Band, Because it has the highest information gain (0.1038).

Manual split check:

H_after = 0.6737 − 0.1038
= 0.5699

* A lower remaining entropy means the split is useful.

5. Python Implementation

The program below generates 240 synthetic records, preprocesses categorical features, trains an entropy-based Decision Tree, evaluates it & prints interpretable tree output.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,
precision_score, recall_score, f1_score,
confusion_matrix
np.random.seed(42)
n = 240
df = pd.DataFrame({
"PH":
np.random.choice(["Acidic", "Neutral", "Alkaline"], n),
"Turbidity": np.random.choice(["Low", "High"],
n), "TDS": np.random.choice(["Low", "Medium",
"High"], n),
"Chlorine": np.random.choice(["Adequate", "Low"],
```

```
"Microbial": np.random.choice(["Absent", "Present"], n),
"Source": np.random.choice(["protected", "Community"
"Surface"], n) })
X = df.drop("class", axis=1)
y = df["class"]
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size = 0.20, random_state = 42)
preprocess = columnTransformer([
    ("cat", OneHotEncoder(handle_unknown
= "ignore"), X.columns)])
model = pipeline([("preprocess", preprocess),
        ("tree", DecisionTreeclassifier(
          criterion = "entropy", max_depth=6,
          random_states = 42 ))
                ])
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("confusion matrix: \n")
confusion_matrix(y_test, y_pred)
print("Accuracy: ", accuracy_score(y_test, y_pred))
print("precision: ", precision_score(y_test, y_pred,
pos_label="safe"))
print("Recall:", recall_score(y_test, y_pred, pos_la
-bel="safe"))
print("F1_score:", f1_score(y_test, y_pred, pos_
label="safe"))
```

...Hot Encoded

6. Results and Evaluation

Dataset size = 240 records. Train/test split = 80%/20%, giving 192 Training & 48 test records.

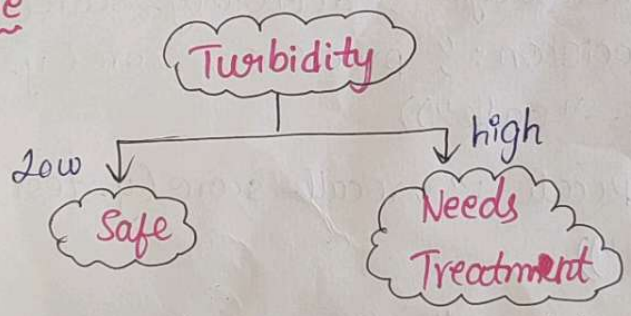
Confusion matrix (row = actual, column = predicted)

	Predicted safe	Predicted Needs Treatment
Actual Safe	45	0
Actual Needs Treatment	0	3

Metric	Result
Accuracy	1.0
Precision	1.0
Recall	1.0
F1-score	1.0

Interpretation: The test split produced zero false-safe predictions & zero false alerts. For a safety-oriented screening system, false safe errors are especially important. So, this results is encouraging but cannot certify water safety.

7. Decision Tree



8. Comparison: version space Vs Decision Tree

version space:

Decision Tree:

☆ Uses Candidate Elimination ☆ Uses Entropy & information gain
☆ Find S & G boundaries ☆ Creates decision rules
☆ Simple & easy to understand ☆ More flexible
☆ Needs fewer examples ☆ works well with larger datasets

Conclusion: Version space explains the concept boundary, while Decision Tree is better for practical classification

9. SDG 6 & SDG 3 Analysis

SDG 6 - clean water & sanitation

This project supports SDG 6 by providing a simple Machine learning Approach to screen water samples. It can help identify samples that may require treatment.

SDG 3 - Good Health and well-Being

This project supports SDG 3 because identifying potentially unsafe water can help reduce exposure to water-related health risks.

Risks

False safe: Unsafe water may be classified as safe.
False Alert: Safe water may be classified as Needs Treatment.
The model cannot replace laboratory testing.

Limitations:

Dataset is Synthetic
only a small no. of features are used.
water Quality can change with location & season
Categorical bands may lose detailed information.
Model accuracy may differ on real-world data.

10. Conclusion :

This project shows how Machine learning can be used to screen water quality. Candidate Elimination helps identify the concept boundary, while the Decision Tree uses Entropy & information gain to make decisions. The model classifies water samples as safe or Needs Treatment. It is simple & easy to understand, but the dataset is synthetic may not represent all real-world conditions. Overall the project demonstrates the useful application of Machine Learning for clean & safe water Screening.